

C++ Projet



Marc Fonvieille et Joël Gay

1 Généralités

L'objectif de ce projet est de mettre en œuvre les connaissances que vous avez acquises durant l'enseignement de la programmation orientée objet appliquée au langage C++. Pour ce faire vous aller devoir :

- Comprendre le cahier des charges
- Modéliser ce cahier des charges sous la forme d'un diagramme de classes en réfléchissant à l'implémentation de chaque fonctionnalité
- Commencer enfin l'étape de programmation
- Préparer un **Makefile** pour permettre de compiler votre programme en compilation séparé.

Ce projet **est orienté objet**, c'est-à-dire qu'il vous est demandé de mettre en pratique les notions vues en cours (héritage, encapsulation, classes, polymorphisme, surcharge, etc.).

Le **rapport** est à rendre au plus tard le **vendredi 30 octobre 2026 à 23H59 (Heure de Paris) par mail aux deux enseignants**. Le projet aura une séance de soutenance le **mardi 3 novembre matin** suite à laquelle le code devra être rendu. Tout retard sera pénalisé. Une séance de coding de modification (Cf. explications dans la partie Évaluation et rendus) est prévue le **vendredi 6 novembre** après le DS.

Vous continuerez à travailler sur les exercices de TP **en parallèle** avec le projet. A l'issue de la dernière séance, vous aurez toujours la possibilité de poser des questions par mail. Ce projet doit être réalisé **en binôme** (il peut en **monôme**, mais c'est *fortement* déconseillé). Aucun groupe de taille plus grande ne sera accepté. Dans le cas d'un binôme, le travail de chacun devra être apparent dans le rapport mais également dans le code et lors de la soutenance.

Le sujet du projet a été rédigé de manière à décrire les fonctionnalités basiques du système. Nous proposons ensuite toute une série de fonctionnalités supplémentaires. De plus, il n'y a

pas de précisions concernant l'implémentation ou la structure interne du projet qui vous sont laissées au choix.

2 Sujet - Le jeu

L'objectif de ce projet est de créer une version du jeu de société de type deckbuilding Dominion. Il est produit par Rio Grande Game. Pour les joueurs, le site web Dominion Games permet d'y jouer en plus d'avoir un tutoriel, cette vidéo vous donne une explication en 5 min. La description ci-dessous provient de ce site.

2.1 Introduction

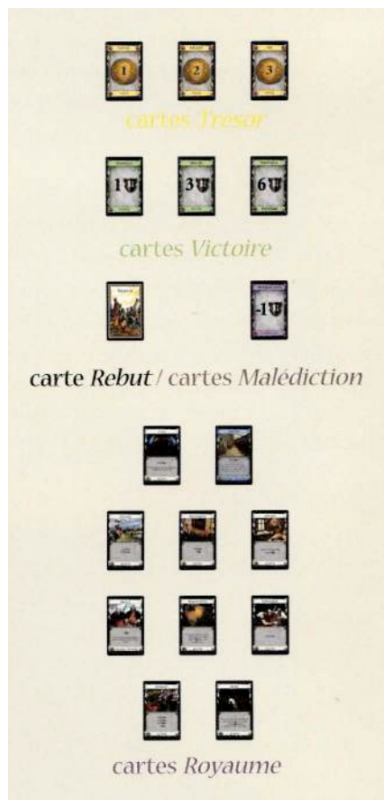
Dominion est un jeu où vous construisez un *deck* qui représente votre royaume. Il est constitué de vos ressources, de vos points de victoire, et des nombreuses actions que vous pouvez effectuer. Au départ ce n'est qu'un piètre ramassis de propriétés et de pièces de cuivre mais vos ambitions feront en sorte qu'à la fin de la partie votre deck fourmille de pièces d'or, de provinces et de nombreux citoyens et constructions qui constituent votre château et royaume.

Dans la boîte de base voici comment se déroule la partie.

Le jeu est axé sur un principe très simple : constituer un deck victorieux qui servira et s'enrichira tout au long de la partie. Les cartes sont de trois catégories :

- *Les cartes Victoire* : *Domaines*, *Duchés* ou *Provinces*, ce sont des cartes qui n'ont qu'une fonction : rapporter des points de victoire en fin de partie. Elles "pollueront" donc le deck dès qu'elles y seront ajoutées, mais sont nécessaires pour gagner.
- *Les cartes Royaume* : Ce sont les cartes action du jeu, (le jeu de base en comporte 25 différentes), elles apportent chacune leur lot de possibilité et permettent des "combos".
- *Les cartes Trésor* : Comme souvent, l'argent c'est le nerf de la guerre, les cartes Trésor vous permettent d'acquérir de nouvelles cartes et ainsi d'étoffer votre deck.

La préparation demande en premier lieu de **choisir les 10 cartes Royaume qui seront en jeu**, (les autres ne serviront pas et resteront dans la boîte). Les 10 exemplaires (12 pour les *jardins*) des cartes choisies sont placés en piles, face visible, sur la table. Les cartes pourront être achetées par les joueurs, tant qu'elles seront disponibles. Voici la configuration initiale sur la table :



Chaque joueur commence dans son deck avec :

- 7 cartes de **cuivre**, valant 1 d'achat ;
- 3 cartes de **domaine**, donnant chacune un point de victoire.

Le Deck de départ de chaque joueur comprend 7 cartes cuivre (valeur 1) et 3 cartes Domaine (1PV)



On place ensuite les cartes Trésor (*cuivre*, *argent* et *or*), les cartes Victoire (12 *domaines*, 12 *duchés* et 12 *provinces* à 3 ou 4 joueurs ; contre 8 à 2 joueurs), ainsi que les cartes *Malédiction*, en piles, face visible.

Le terme *Rebut* désignera l'emplacement où seront placées les cartes écartées définitivement durant la partie.

2.2 Le Tour de Jeu

Chaque joueur, à son tour, joue les 3 phases qui constituent un tour de jeu :

1. La Phase Action : Durant cette phase le joueur peut jouer, s'il en a au moins une en main, une carte Action. La carte jouée est résolue immédiatement et complètement avant d'éventuellement en jouer une autre (si la carte jouée vous le permet). Les cartes actions permettent :
 - de jouer X actions supplémentaires pour ce tour.
 - de piocher immédiatement X cartes de votre deck.
 - de disposer de X pièces supplémentaires pour votre phase d'achat de ce tour.
 - de disposer d'un achat supplémentaire pour ce tour.
 - de défausser des cartes de votre main dans votre défausse personnelle.
 - de mettre une carte au rebut (elle est donc *écartée* définitivement de la partie).
 - de recevoir une carte, soit dans votre main, soit sur votre pile de défausse.
 - de mettre des cartes de côté, jusqu'à ce que les instructions indiquent où les mettre.
2. La Phase Achat : Lors de cette phase le joueur peut acheter une carte de la réserve (parmi les cartes Trésor, Victoire et Royaume, voire même Malédiction). Si le joueur a joué, durant sa phase action, des cartes lui permettant d'augmenter son nombre d'achat, il pourra acheter plusieurs cartes. Les cartes ainsi acquises sont placées dans la pile de défausse du joueur.

Si vous avez la possibilité de faire plusieurs achats, vous devez les faire en combinant les cartes Trésor ainsi que les pièces supplémentaires dues aux cartes Action jouées ce tour.

Note : Il est possible de faire un achat à 0 (les cartes *cuiivre* par exemple).
3. La Phase Ajustement : en plus cartes acquises ce tour qui s'y trouvent déjà, le joueur place toutes les cartes jouées dans les phases Action et Achat (Royaume ou Trésor) ainsi que les cartes qu'il a encore en main, dans sa défausse. Le joueur pioche ensuite **5 nouvelles cartes** de son deck, pour former sa nouvelle main (s'il n'y a plus assez de cartes dans son deck, il mélange les cartes de sa défausse pour reformer son deck et y pioche les cartes manquantes).

Exemple :

La main de cartes du joueur :



1 - Phase Action

Le joueur choisit de jouer le village et pioche immédiatement une carte (un domaine). Il dispose de 2 actions supplémentaires :

- Il joue sa carte douves pour piocher immédiatement 2 cartes, une carte argent et une carte bûcheron.
- Il joue sa carte Bûcheron qui lui donne un achat supplémentaire ainsi que +2 pièces pour sa phase d'achat.

Phase Action



2 - Phase Achat

Le joueur choisit de faire l'acquisition d'une carte or pour ses 4 + 2 pièces et d'une carte cuivre pour son achat supplémentaire (coût : 0).

Phase Achat

+1 Achat

2.3 Fin de la Partie

La partie se termine à la fin du tour d'un joueur, lorsque :

- La réserve de cartes *Province* est vide.
- OU 3 piles de la réserve sont vides (la pile malédiction compte).

Les joueurs comptent alors les points de victoire qui se trouvent sur les cartes de leur deck, celui qui en a le plus gagne.

2.4 Quelques exemples de cartes



Permet de piocher 2 cartes supplémentaires et de jouer une autre action pendant sa phase Action.



Permet d'écarter une carte Trésor de sa main pour recevoir une carte Trésor valant 3 de plus. La carte reçue est immédiatement utilisable.



Tous vos adversaires dévoilent les 2 premières cartes de leur deck. S'ils dévoilent des cartes Trésor, ils en écartent une de votre choix. Parmi les cartes ainsi écartées, vous pouvez récupérer celles de votre choix.



Permet de jouer une action supplémentaire et de défausser autant de cartes que désiré. Pour chaque carte défaussée, le joueur en pioche une autre dans son deck.



Permet de piocher 1 carte supplémentaire et de jouer deux autres actions pendant sa phase Action.



Permet de piocher 2 cartes supplémentaires et protège contre une attaque jouée par un adversaire. Dans ce cas il suffit de dévoiler la carte Doutes de sa main pour que l'attaque n'ait pas d'effet (sur le joueur qui dévoile sa carte Doutes).



Permet, pendant la phase d'achat, de disposer d'un achat supplémentaire ainsi que de 2 pièces de plus.



En fin de partie, pour chaque groupe de 10 cartes présentes dans le deck, la carte Jardins rapporte 1 point de victoire. Les cartes Jardins sont cumulables).

2.5 Lire plus

Pour voir plus, on vous donne les règles officielles du jeu, et on rappelle le site web Dominion Games pour y jouer.

3 Évaluation et rendus

L'évaluation comportera trois volets : l'évaluation et présentation du code, celle d'un rapport succinct et enfin le codage de modifications de votre code.

3.1 Rapport

Le rapport doit être envoyé pour présenter le projet. Ce **rapport succinct** prendra la forme d'un texte **au format PDF**, aucun autre format ne sera accepté ou lu. Les éléments qui suivent doivent y être présents peu importe le nombre de pages (cela devrait faire autour d'une dizaines de pages sans doute vu les éléments demandés). Il contient notamment :

- Une explication des choix de conception pour répondre au cahier des charges.
- Le diagramme des classes avec commentaires sur le rôle des classes.
- Pour les binômes, une répartition précise du travail
- Un guide d'utilisation du **Makefile** et explication en cas de particularités
- Une explication succincte de chaque fonctionnalité supplémentaire réalisée (suggérée par le sujet ou de votre propre ajout)
- Les difficultés rencontrées, et comment vous les avez surmontés - ou en quoi ils vous ont empêché de terminer une partie du projet.

Pensez à inclure des captures d'écran de votre programme en fonctionnement (en annexe si vous le souhaitez).

En outre, le soin accordé aux schémas, à la présentation, à l'orthographe, sont pris en compte.

3.2 Soutenance

La **soutenance** est une démonstration de code de 10 à 15 minutes (en fonction du nombre de groupes, vos enseignants vous enverrons un timing). Pendant la démonstration, les enseignants testeront votre code, chercheront les limites, poseront des questions de modélisation. Il vous faut donc simultanément pouvoir montrer les fonctionnalités implémentées, mais aussi pouvoir justifier des choix et expliquer le codage de l'application. C'est l'occasion de montrer aussi tout ce que vous avez fait en plus de la base du jeu.

Suite à la soutenance, le code devra être rendu. Tout retard sera pénalisé.

Votre travail (code + rapport) doit être rendu sous forme d'une **archive ZIP**. Vérifiez que le code envoyé inclut seulement les fichiers **.cpp**, **.h(pp)** et **.txt** avec les données nécessaires au fonctionnement de votre programme, ainsi que le **Makefile**. Tout **CMake** sera refusé ! Un programme ne pouvant être compilé chez les enseignants (Linux) et sur les sessions Linux de l'école ne sera pas accepté.

3.3 Cahier des charges de base

Le projet doit être indépendant de la plateforme utilisée. Il doit pouvoir être compilé par `g++` sans warning (et évidemment sans erreur) et avec le paramètre `-std=c++11`. Vous fournirez obligatoirement un `Makefile` qui permettra de lancer la compilation facilement. Le respect des fonctionnalités imposées et la qualité du code (structure, utilisation **pertinente** d'un maximum de concepts - héritage, polymorphisme etc. - vus en cours et en TP, commentaires) seront primordiaux.

On sépare les besoins d'architecture du code, des besoins fonctionnels.

Infrastructure du projet :

- Un fichier `Makefile` pour compiler votre programme ;
- Chaque classe est dans un fichier à part, et a son fichier de header ;
- Un fichier `main.cpp` qui lance le jeu ;
- Dans le rapport un guide d'utilisation de la compilation, notamment s'il y a plusieurs modes possibles

Fonctionnalités du jeu :

Une version minimale du projet sera de permettre de jouer entre deux joueurs sur une console, à tour de rôle, en respectant les règles énoncées précédemment. Dans un premier temps, comme tout se fait sur la même console, les cartes en main peuvent être connues de tous afin de simplifier. Et on jouera à deux joueurs, avec les dix cartes : atelier, bûcheron, chapelle, douve, festin, jardins, laboratoire, sorcière, village, voleur.

*(Bien sûr il faut donc avoir aussi prévu les 3 cartes trésors (60 **cuivre**, 40 **argent**, 30 **or** initialement), les cartes victoires (24 **domaine**, 12 **Duché** et 8/12 **Province** (8 à deux joueurs) et les 30 **malédiction** (qui font perdre un point de victoire en fin de partie)).*

Il faudra donc :

- La base du jeu fonctionnelle :
 - La possibilité de jouer à 2 joueurs en initialisant correctement leur deck initial
 - Mettre à disposition les cartes achetables pour ce tour.
 - Pouvoir mélanger les decks avant la partie, avant de distribuer 5 cartes chacun.
 - Mettre en place chaque phase de jeu.
 - Surveiller la fin de partie et annoncer la victoire.
- Un mode rapide que l'on peut sélectionner au départ du jeu qui permet :
 - De mettre tous les paquets de la réserve à 1 carte dès le début de la partie afin d'accélérer la fin de partie

L'affichage pourra être assez minimaliste sur la console.

3.4 Extensions

3.4.1 Extension à faire en premier

Avant de penser à tout le reste, faite en sorte de faire toutes les cartes de la base. Cela permettra alors de créer des configurations plus intéressantes. Les cartes en question sont décrites dans le site officiel et des détails sont mis dans le pdf des règles que vous avez. **Attention** : nous supposons travailler sur la première édition française du jeu, dont certaines cartes ont disparu dans la seconde édition.

3.4.2 Extensions supplémentaires

Nous proposons dans cette section plusieurs possibilités d'améliorations, de niveaux de difficultés variables, et qui chacune peuvent mettre en place différentes notions que vous avez apprises dans ce module.

Pour chaque extension proposée, on vous demandera dans le rapport de la décrire succinctement et la façon de la réaliser.

Voici ici un résumé de quelques possibilités d'extensions, avec une estimation de leur niveau de difficulté, mais sans ordre de préférence :

Nom	Descriptif et idées	Difficulté
Choix des dix cartes royaumes	Les dix cartes royaumes peuvent être sélectionnées au début, soit parmi plusieurs sets de base, soit nominativement, soit aléatoirement	★
IA	Créer une IA contre laquelle jouer, à 2, 3 ou 4 joueurs potentiellement. L'IA peut être assez complexe et prendre en compte tous les mécanismes du jeu	★ à ★★★
Interface graphique	Log des actions, couleur de console, affichage d'une forme de terrain de jeu (ascii art, rafraichissement console), utilisation bibliothèque graphique, etc.	★ à ★★★
Plus de deux joueurs	Possibilité de jouer à plus de 2 joueurs, en console ou en ligne	★ à ★★★
En ligne	Possibilité de jouer à distance	★★★
Sauvegarde de partie	Possibilité de s'arrêter en cours de partie, et de revenir	★★
Extensions du jeu	Possibilité d'ajouter des sets de cartes supplémentaires de toutes les extensions du jeu (cf. sur le site de jeu et site officiel)	★ à ★★★

Rappel : votre code doit se compiler et s'exécuter sur les machines de l'école, il faut donc éviter les bibliothèques *exotiques*

3.5 Séance de coding de modification

L'objectif de cette séance est de pouvoir évaluer votre maîtrise du code que vous nous aurez présenté lors de la soutenance en vous demandant, individuellement, d'effectuer un certain nombre de modifications.

Organisation :

- durée : 1h (1h20 tiers temps).
- individuellement sur un ordinateur de l'école sans aucun accès réseau.
- vous devrez venir avec une clé USB **personnelle** avec votre code dessus (celui présenté lors de la soutenance ou éventuellement une version améliorée entre temps) **qui n'a besoin d'aucune librairie supplémentaire**. Vous êtes responsable du bon fonctionnement de départ du code présent sur la clé USB. Sur la clé USB, il sera possible d'avoir la version numérique de votre cours.
- Votre projet doit compiler avec un Makefile sur l'environnement Linux de l'école.
- Vous aurez une ou plusieurs modifications ou améliorations à effectuer à choisir parmi un ensemble de modifications qui vous sera proposé
- Le mode de jeu utilisé pour ces modifications sera le mode le 1vs1 joueur contre joueur.
- À la fin de l'heure de codage, nous validerons avec vous les modifications. Si à ce moment la compilation ne fonctionne pas ou l'exécution ne fonctionne pas, la note sera de 0 à cette partie. Les warnings seront acceptés (pensez à supprimer **-Werror** de votre Makefile à cet effet).

Le poids de cette séance sera d'environ 33% de la note totale du projet, le reste portant sur le rapport et la soutenance.

Attention : en binôme, vous vous êtes forcément réparti les tâches de codage. Il faudra, cependant, être capable d'intervenir sur les parties de code développées par votre binôme.